



Importance of Software

Architecture in System Design

Software architecture is the backbone of system design, ensuring scalability, maintainability, security, cost efficiency, performance and future adaptability. Here's a list of key reasons why software architecture is critical in system design:

Defines system structure & organisation: Provides a blueprint for organising components, modules, and interactions. Example: Layered architecture separates UI, business logic, and data access for clarity.

Ensures scalability: Supports horizontal/vertical scaling to handle growing user loads. Example: Microservices allow independent scaling of services (e.g., Auth Service vs. Payment Service).

Improves maintainability: Modular design makes it easier to update, debug, and enhance the system. Example: Using OOP principles (encapsulation) to isolate changes.

Enhance performance: Optimises resource usage (CPU, memory, network) through efficient design. Example: Caching layers in architecture reduce database load.

Enables reusability: Promotes reusable components (e.g., libraries, APIs) across projects. Example: A shared AuthenticationModule used in multiple apps.

Facilitates team collaboration: Clear architecture acts as a shared reference for developers, reducing conflicts. Example: UML diagrams standardise communication in Agile teams.

Supports security: Builds security into the design (e.g., encryption, access control layers). Example: API gateways validate requests before reaching core services.

Reduces development costs: Prevents costly redesigns by addressing requirements early. Example: Choosing serverless architecture to minimise infrastructure costs.



Ensures reliability & fault tolerance: Designs for failures (e.g., retry mechanisms, redundancy).

Example: Circuit breakers in microservices prevent cascading failures.

Simplifies integration: Standardises how systems interact (APIs, message queues). Example: RESTful APIs enable seamless integration with third-party services.